

Pure Pandrosion vectorized via exact telescopic slopes

Mathematical and experimental study of script 115

Ivan Besevic

April 29, 2026

Abstract

We study the engine `115_pandrosion_vectorized_pure_thales_engine.py`. Unlike engine 113, which uses a Jacobian-type local corrector, 115 builds an exact slope matrix $Q(a, b)$ satisfying

$$F(b) - F(a) = Q(a, b)(b - a)$$

via a monomial telescopic identity. This construction is derivative-free at the corrector level: no Jacobian is built for the local correction. The cost of $Q(a, b)$ is reduced through vectorization: tabulated powers, prefix/suffix factors, and a NumPy/BLAS matrix product. The engine retains the same single geometric flow as 113: Riemann-Mobius, Thales-type homothety, starting-point optimization, then correction. On the available tests, 115 recovers 144/144 roots for $ks(2, 12)$, 196/196 for $ks(2, 14)$, 64/64 for $ks(3, 4)$, 81/81 for $ks(4, 3)$, and a strict root for $ks(6, 16)$. Compared to 114, vectorization speeds up the construction of Q by roughly $6\times$ to $7\times$ on the 2, 12 and 2, 14 cases. We also describe follow-up script 116, which tests the same local engine on multiple polynomial families, and script 117, which exports identical systems to external homotopy solvers and provides a budgeted local SciPy baseline. These comparisons are preliminary: Bertini, PHCpack, HomotopyContinuation.jl, and a Lairez-style solver are treated as mandatory future baselines, but they were not installed in the present run environment.

1 Motivation

The numerical solution of square polynomial systems is already a mature subject. Total-degree and polyhedral homotopy continuation, as implemented in systems such as Bertini, PHCpack, and HomotopyContinuation.jl, is the standard reference point when the goal is to compute all isolated complex solutions [1, 7, 2]. The present work is therefore not proposed as a replacement for numerical algebraic geometry. It studies a narrower question: whether a direct geometric multistart method, combined with an exact finite slope identity, can extract useful roots of high-degree systems without explicitly tracking the full Bezout path set.

Version 113 showed that a powerful geometric flow can extract every root of certain Kostlan systems without explicitly tracking all homotopy paths. But its local corrector solves

$$J_G(y)\Delta = -G(y).$$

Version 115 replaces this step with an exact Pandrosion slope. The goal is to keep the global efficiency of the Thales-Riemann flow while making the local correction faithful to the geometric identity

$$G(b) - G(a) = Q_G(a, b)(b - a).$$

Avoiding the Jacobian is not claimed to make the method automatically faster or more reliable. The point is structural: $Q_G(a, b)$ is a non-infinitesimal finite slope, so the local update uses information between two separated points rather than the derivative at a single point. This makes 115 closer to the original Pandrosion philosophy, at the cost of building a more expensive matrix.

2 Related work and baselines

The natural comparison class is numerical homotopy continuation. The book of Sommese and Wampler [6] and the Bertini reference [1] give the standard framework: construct a homotopy from an easier start system, track paths, and certify or refine approximate isolated solutions. PHCpack [7] emphasizes black-box solving and sparse structure. HomotopyContinuation.jl [2] provides a modern Julia implementation and exposes monodromy, parameter homotopies, path tracking, and certification interfaces.

The complexity-theoretic background is the Shub-Smale program on the complexity of Bezout’s theorem [5]. Lairez’s deterministic algorithm for Smale’s 17th problem [3] gives an average-polynomial-time result for computing an approximate root in the Blum-Shub-Smale model with square root. This is a theoretical baseline of a different kind from Bertini or PHCpack: it addresses the existence of a deterministic average-polynomial algorithm rather than a drop-in command-line solver used in the present experiments.

Script 117 was added after this review precisely to make these comparisons reproducible. It exports the same systems generated by script 116 to Bertini, PHCpack, and HomotopyContinuation.jl input formats; it also contains a configurable hook for a Lairez-style implementation. In the current machine, the commands `bertini`, `phc`, and `julia` were not installed, so those comparisons are reported as skipped rather than replaced by proxy numbers.

3 Polynomial systems and change of coordinates

Let

$$F = (F_1, \dots, F_n) : \mathbb{C}^n \rightarrow \mathbb{C}^n$$

be a dense polynomial system. The engine uses a linear map

$$z = Ay$$

and solves

$$G(y) = F(Ay) = 0.$$

As in 113, the initial point is generated by a single flow:

$$y_j = \Lambda p_j \frac{\cos(\theta_j)(u_j/p_j) + \sin(\theta_j)}{-\sin(\theta_j)(u_j/p_j) + \cos(\theta_j)}. \quad (1)$$

Here $u \in \mathbb{C}^n$ is a deterministic complex direction, p_j is a nonzero pole coordinate, θ_j is a Riemann-Mobius angle, and $\Lambda > 0$ is a Thales homothety scale chosen from a wide power ladder. This map blends homothety, projective inversion, and Riemann rotation in a single formula. It is a heuristic initialization layer, not a theorem guaranteeing basin entry for every system.

4 Pandrosion slope matrix

Definition 1. Let $a, b \in \mathbb{C}^n$. A matrix $Q_F(a, b) \in \mathbb{C}^{n \times n}$ is a Pandrosion slope for F between a and b if

$$F(b) - F(a) = Q_F(a, b)(b - a). \quad (2)$$

For the transformed system $G(y) = F(Ay)$, we set

$$Q_G(a_y, b_y) = Q_F(Aa_y, Ab_y)A. \quad (3)$$

Then

$$G(b_y) - G(a_y) = Q_G(a_y, b_y)(b_y - a_y).$$

5 Monomial telescopic identity

Consider a monomial

$$m(z) = z^\alpha = \prod_{k=1}^n z_k^{\alpha_k}.$$

For each coordinate j , define

$$S_{j,\alpha_j}(a_j, b_j) = \begin{cases} \sum_{r=0}^{\alpha_j-1} b_j^{\alpha_j-1-r} a_j^r, & \alpha_j > 0, \\ 0, & \alpha_j = 0. \end{cases} \quad (4)$$

Lemma 1 (telescopic slope of a monomial). *We have*

$$m(b) - m(a) = \sum_{j=1}^n \left(\prod_{k < j} b_k^{\alpha_k} \right) S_{j,\alpha_j}(a_j, b_j) \left(\prod_{k > j} a_k^{\alpha_k} \right) (b_j - a_j). \quad (5)$$

Proof. We write the total difference by successively replacing the coordinates of a by those of b :

$$\prod_k b_k^{\alpha_k} - \prod_k a_k^{\alpha_k} = \sum_j \left(\prod_{k < j} b_k^{\alpha_k} \right) (b_j^{\alpha_j} - a_j^{\alpha_j}) \left(\prod_{k > j} a_k^{\alpha_k} \right).$$

Now

$$b_j^q - a_j^q = (b_j - a_j) \sum_{r=0}^{q-1} b_j^{q-1-r} a_j^r.$$

Substituting this identity yields (5). □

For a polynomial

$$F_i(z) = \sum_{\alpha} c_{i,\alpha} z^\alpha,$$

column j of $Q_F(a, b)$ is obtained by summing the contributions (5) weighted by $c_{i,\alpha}$. This construction is exact, up to floating-point error, and requires no derivatives.

6 Pure Pandrosion corrector

At the current iterate a , the engine builds a second point b via a non-infinitesimal geometric formula. It then computes $Q_G(a, b)$ and solves

$$Q_G(a, b)\Delta = -G(a), \quad a_+ = a + \lambda\Delta. \quad (6)$$

The parameter λ is chosen by a proportional search that requires a decrease of the residual. This search gives an acceptance rule for one update; it is not, by itself, a convergence theorem. If the finite slope is ill-conditioned, if the endpoint b is poorly placed, or if the line search cannot reduce the residual within its finite budget, the trial is marked as a failure. There is no Newton fallback, no least squares, no bivariate specialization, and no backup homotopy.

Remark 1. *The engine still uses `np.linalg.solve`, but only to solve the dense linear system (6). The distinction is algorithmic rather than rhetorical: the local linearization matrix is not a Jacobian and is built from the exact finite identity (2). In complexity terms, each accepted or rejected correction epoch pays for power tables, vectorized monomial products, a dense matrix product CT , and one $n \times n$ linear solve.*

7 Vectorization of $Q(a, b)$

Version 114 built $Q(a, b)$ via Python loops. Engine 115 vectorizes the computation.

Let $E \in \mathbb{N}^{M \times n}$ be the matrix of exponents of the M monomials, and let $C \in \mathbb{C}^{n \times M}$ be the matrix of coefficients. For points a, b , we tabulate

$$a_j^m, quad b_j^m, \quad m = 0, \dots, d.$$

Then we build, for all monomials:

$$P_j^<(b) = \prod_{k < j} b_k^{E_{\cdot k}}, \quad (7)$$

$$P_j^>(a) = \prod_{k > j} a_k^{E_{\cdot k}}, \quad (8)$$

$$T_{\cdot j} = P_j^<(b) P_j^>(a) S_{j, E_{\cdot j}}(a_j, b_j). \quad (9)$$

The slope matrix is then obtained by a dense product:

$$Q_F(a, b) = CT. \quad (10)$$

The table $S_{j, m}$ is computed via the recurrence

$$S_{j, 0} = 0, \quad S_{j, m} = b_j^{m-1} + a_j S_{j, m-1}. \quad (11)$$

This recurrence is equivalent to (4).

8 Algorithm

Input: F , a trial budget P , a target count N .

Output: distinct numerical roots.

1. Generate the dense system and the matrices E, C .
2. For each trial, produce u, Λ, θ, p , then y via (1).
3. Improve the starting point via startopt.
4. Repeat:
 - (a) choose a geometric point b associated with a ;
 - (b) build $Q_G(a, b)$ via (3) and (10);
 - (c) solve $Q_G(a, b)\Delta = -G(a)$;
 - (d) apply a proportional search on $a + \lambda\Delta$.
5. Convert $z = Ay$, validate $\|F(z)\|$, then deduplicate.

The design choices are pragmatic. The Mobius-Thales layer searches a large family of geometric scales without enumerating homotopy paths. StartOpt performs a cheap local radial improvement before the expensive slope construction. Deduplication is necessary because the same numerical root is often reached from several starts. A failed trial is not catastrophic; the method is designed as a Las-Vegas-style extractor of useful roots rather than as a certified path tracker.

Case	Bezout	Roots requested	Unique roots	Trials	Failures	Time (s)
$ks(2, 12)$	144	144	144	272	124	5.345
$ks(2, 14)$	196	196	196	390	192	5.967
$ks(3, 4)$	64	64	64	125	20	1.586
$ks(4, 3)$	81	81	81	220	57	3.464
$ks(6, 16)$	16,777,216	1	1	1	0	2.639

Table 1: Main results of engine 115.

Case	Q calls	Q time (s)	Max residual	Mean residual
$ks(2, 12)$	6,287	1.024	$9.77 \cdot 10^{-9}$	$3.79 \cdot 10^{-9}$
$ks(2, 14)$	8,624	1.154	$9.96 \cdot 10^{-9}$	$3.99 \cdot 10^{-9}$
$ks(3, 4)$	2,046	0.263	$9.91 \cdot 10^{-9}$	$4.21 \cdot 10^{-9}$
$ks(4, 3)$	3,950	0.655	$9.86 \cdot 10^{-9}$	$4.17 \cdot 10^{-9}$
$ks(6, 16)$	16	1.670	$6.15 \cdot 10^{-13}$	$6.15 \cdot 10^{-13}$

Table 2: Slope statistics and residuals for 115.

9 Experimental results

9.1 Failure accounting

The column “Failures” in Table 1 counts starts that did not produce a new validated root. This includes genuine non-convergence, no-decrease line-search exits, timeouts when enabled, and starts landing in regions that eventually duplicate a previously found root. The failure rate is therefore not directly a condition number or basin-volume estimate. For example, $ks(2, 12)$ uses 272 trials to obtain 144 distinct roots, with 124 failures. This is expensive compared with an ideal one-start-per-root method, but still far below a full enumeration of arbitrary dense-path starts when only the final distinct roots are needed.

The $ks(6, 16)$ line should be read differently from the full-coverage rows. Its Bezout number is $16^6 = 16,777,216$, and the experiment requested only one root. The result demonstrates that the geometric starts and pure slope corrector can find a strict root in a very high Bezout ambient space; it does not claim to enumerate, sample uniformly, or certify a positive fraction of all roots.

10 Comparison with 114

Version 114 used the same telescopic slope idea, but with a less vectorized construction. The available measurements give:

Case	Time 114 (s)	Time 115 (s)	Total speedup	Q speedup
$ks(2, 12)$	11.468	5.230	$2.19\times$	$6.02\times$
$ks(2, 14)$	17.457	6.735	$2.59\times$	$7.43\times$

Table 3: Effect of vectorization compared with 114.

11 Multifamily and budgeted baseline

Script 116 extends the 115 engine by replacing the single Kostlan generator with a family layer:

ks	ks_sparse	dense_iid	sparse_iid
fewnomial	degree_shell_ks	mixed_degree	real_ks
phase_ks	diagonal	chain	cyclic
ill_scaled			

This gives a broader stress test than dense Kostlan systems alone. On $ks(2,10)$, script 116 with `-families all` found 104/104 requested roots across all 13 families. On the multivariate test $ks(3,4)$, with `-families all`, `-count 4`, and `-pool 512`, it found 48/52 requested roots: 12 of the 13 families reached the requested count, while `ill_scaled` failed.

Script 117 then compares the same systems with a local SciPy multistart baseline. This is not a substitute for Bertini, PHCpack, or HomotopyContinuation.jl. It is only a local baseline using `scipy.optimize.root`, with the important correction that it is now budgeted by a global number of evaluations of F . With `-scipy-eval-budget 0`, the SciPy budget is set equal to `-pool`, so the $ks(3,4)$ comparison gives SciPy 512 residual evaluations per family.

Method	Families	Complete	Roots	Budget accounting	Comment
116 Pandrosion	13	12	48/52	variable evaluations; 198-890 on solved families, 23,309 on <code>ill_scaled</code>	best coverage in this local run, but not equalized by evaluation count
SciPy multistart	13	3	38/52	512 evaluations per family	often stopped by budget exhaustion
External homotopy	13	–	–	inputs exported only	commands unavailable in current environment

Table 4: Budgeted local comparison from script 117 on $ks(3,4)$, all 13 families, 4 requested roots per family.

The honest conclusion is mixed. Under the local budgeted SciPy baseline, 116 wins in coverage on the tested multifamily $ks(3,4)$ benchmark. But this is not yet a claim of superiority over homotopy continuation. A publishable comparison must run the exported Bertini, PHCpack, and HomotopyContinuation.jl inputs on the same systems, report wall time, residual quality, number of paths or tracked solutions when available, and certification status when supported.

12 Discussion

Vectorization does not change the mathematical content of the slope. It merely shifts the monomial summation toward matrix-matrix product operations. Thus 115 retains the pure Pandrosion character of 114 while substantially reducing the cost of computing Q . The cost remains higher than a classical Jacobian corrector at small degrees, since $Q(a,b)$ encodes a global slope between two points and not just an infinitesimal linearization.

The present evidence suggests the following interpretation. The method is strong as a root extractor on the tested Kostlan and multifamily instances, especially when full path enumeration is not the objective. Its weak point is robustness under severe scaling: `ill_scaled` breaks the current default parameters in the $ks(3,4)$ multifamily run. This is valuable information, not a nuisance. It indicates that future versions need either scale-aware starts, coefficient normalization strategies, or an adaptive probe construction for $Q(a,b)$.

13 Reproducibility

Main commands:

```
python -S 115_pandrosion_vectorized_pure_thales_engine.py \
```

```

--cases 2,12 --equation-normalize --count 144 --pool 5000 \
--epochs 30 --accept 1e-8 --cluster-sep 1e-8 \
--startopt-candidates 12 --probe-scale 0.03 --line-search 14

python -S 115_pandrosion_vectorized_pure_thales_engine.py \
--cases 2,14 --equation-normalize --count 196 --pool 7000 \
--epochs 30 --accept 1e-8 --cluster-sep 1e-8 \
--startopt-candidates 12 --probe-scale 0.03 --line-search 14

Multifamily and benchmark commands:

.venv/bin/python \
flow/116_pandrosion_multifamily_vectorized_pure_pandrosion.py \
--cases 3,4 --families all --count 4 --pool 512

.venv/bin/python \
flow/117_pandrosion_solver_benchmark.py \
--cases 3,4 --families all --count 4 --pool 512 \
--solvers local --scipy-starts 128

.venv/bin/python \
flow/117_pandrosion_solver_benchmark.py \
--list-solvers

```

The last command documents the available adapters. On the current machine, Bertini, PHC-pack, and Julia were not in the executable path; script 117 therefore generated input files and marked those solvers as skipped.

14 Conclusion

Engine 115 provides an autonomous, multivariate, derivative-free (at the local-corrector level) version of Pandrosion. Its mathematical contribution is the vectorized implementation of the exact slope $Q(a, b)$ arising from the monomial telescopic identity. The results show that one preserves the root coverage obtained by the pure engine 114 while substantially accelerating the computation. Script 116 broadens the test distribution beyond dense Kostlan systems, and script 117 begins the required comparison infrastructure against standard solvers.

The publication-level claim should remain precise. The current evidence supports the usefulness of 115/116 as fast direct root extractors on the tested systems and shows an advantage over a budgeted local SciPy multistart baseline. It does not yet establish competitiveness against Bertini, PHCpack, HomotopyContinuation.jl, or Lairesz-style average-polynomial homotopy algorithms. Those comparisons are now technically prepared by script 117 and should be the next experimental step.

References

- [1] D. J. Bates, J. D. Hauenstein, A. J. Sommese, and C. W. Wampler. *Numerically Solving Polynomial Systems with Bertini*. SIAM, Software, Environments, and Tools 25, 2013.
- [2] P. Breiding and S. Timme. HomotopyContinuation.jl: A package for homotopy continuation in Julia. In *Mathematical Software – ICMS 2018*, Lecture Notes in Computer Science 10931, pages 458–465, Springer, 2018.

- [3] P. Lairez. A deterministic algorithm to compute approximate roots of polynomial systems in polynomial average time. *Foundations of Computational Mathematics*, 17(5):1265–1292, 2017.
- [4] P. Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.
- [5] M. Shub and S. Smale. Complexity of Bezout’s theorem V: polynomial time. *Theoretical Computer Science*, 133(1):141–164, 1994.
- [6] A. J. Sommese and C. W. Wampler. *The Numerical Solution of Systems of Polynomials Arising in Engineering and Science*. World Scientific, 2005.
- [7] J. Verschelde. Algorithm 795: PHCpack: a general-purpose solver for polynomial systems by homotopy continuation. *ACM Transactions on Mathematical Software*, 25(2):251–276, 1999.